

Содержание

Введение	2
Постановка задачи	4
Обзор литературы	5
Глава 1. Методы адаптации языковых моделей	11
1.1. Обзор существующих методов дообучения LLM	11
1.2. Применимость PEFT-методов к задачам стилизации	15
Глава 2. Данные и постановка прикладной задачи	17
2.1. Постановка задачи	17
2.2. Описание датасета	18
Глава 3. Эксперименты и результаты	19
3.1. Методы дообучения	19
3.2. Используемые метрики	20
3.3. Модель ruGPT3-small	21
3.4. Модель Qwen2.5-0.5B	23
3.5. Оценивание через LLM-as-a-Judge	29
Выводы	32
Заключение	33
Список литературы	34
Приложение	38

Введение

Большие языковые модели (LLM) достигли значительных успехов с момента появления архитектуры трансформеров. Одним из наиболее востребованных направлений их применения является генерация текста, в частности, на основе decoder-only моделей, реализованных как Generative Pre-trained Transformer (GPT). Современные LLM эффективно решают задачи автоматизированного создания текстов, включая генерацию ответов на электронные письма и пользовательские запросы.

Вместе с тем генерируемые моделью тексты, как правило, обладают нейтральным или усредненным стилем и недостаточно учитывают индивидуальные особенности коммуникации, характерные для конкретных пользователей или сообществ. Отсутствие стилевой персонализации может снижать восприимчивость текста и его естественность, что особенно критично в задачах человеко-ориентированного взаимодействия, таких как генерация ответов в социальных сетях или чатах поддержки.

Для решения данной проблемы в практических системах часто используется prompt-tuning, позволяющий управлять стилем генерации за счет изменения входного запроса. Данный подход обладает низкой стоимостью внедрения и эффективен в краткосрочной перспективе, однако расходует контекстное окно, увеличивает вычислительные затраты и плохо применим к моделям с ограниченным числом параметров или малой длиной контекстного окна [1].

Альтернативным решением является дообучение модели на примерах нужного стиля, позволяющее усвоить моделью лексические и стилистические закономерности целевых текстов. Данный подход успешно применяется к генеративным состязательным сетям (GAN) при переносе отдельных стилевых признаков [2, 3].

В отличие от GAN, обычно оперирующих бинарными стилевыми противопоставлениями («формально / неформально», «позитивно / негативно») современные модели, основанные на трансформерах, содержат большее число параметров и способны улавливать более сложные паттерны. Рассматриваемая в данной работе задача заключается в обучении нейронной сети перенимать не бинарный признак, а коллективный стиль сообщества – совокупность

характерных слов, оборотов и синтаксических конструкций, формируемую множеством авторов и развивающуюся с течением времени. Практическая ценность работы заключается в том, что ее результаты позволяют стилизовать языковые модели не под узкие специализированные задачи, а под широкий спектр сценариев.

Постановка задачи

Целью данной работы является изучить применимость методов полного и параметро-эффективного дообучения языковых моделей к задаче стиливой адаптации текстовой генерации на русскоязычном корпусе комментариев из Telegram.

Для достижения поставленной цели были сформулированы следующие задачи:

- провести обзор существующих подходов к нейросетевой стилизации текста и методов дообучения языковых моделей, в том числе параметро-эффективные методы (PEFT);
- собрать, предобработать и опубликовать датасет для задачи стиливой адаптации коллективного стиля на основе русскоязычных Telegram-каналов;
- формализовать постановку задачи стиливой адаптации в терминах казуального языкового моделирования;
- провести эксперименты по полному дообучению и методам LoRA и (IA)³ на больших языковых моделях и исследовать влияние применения LoRA к различным группам слоёв трансформера;
- разработать набор метрик для оценки качества стиливой адаптации, лексического разнообразия и устойчивости к катастрофическому забыванию;
- провести сравнительный анализ методов и проинтерпретировать результаты;
- реализовать метод оценки стиливого соответствия на основе подхода LLM-as-a-Judge.

Обзор литературы

В настоящем разделе рассматриваются существующие подходы к задаче стилевой адаптации текста: приводится формальная постановка задачи, описывается эволюция методов и обосновывается выбор подхода, принятого в данной работе.

Задача стилевой адаптации

Задача стилевой адаптации текста (text style transfer, TST) состоит в изменении стилистических характеристик текста при сохранении его семантического содержания [4]. К таким характеристикам относят тональность, формальность, вежливость, идиостиль автора и иные лингвистические атрибуты, не связанные напрямую с передаваемым смыслом. Типичными примерами служат преобразование негативного отзыва в позитивный, переписывание неформального текста в деловом регистре или детоксификация агрессивных высказываний.

Применение нейросетевых подходов в TST началось в 2017 году с работами [6] и [3], после чего область претерпела несколько смен парадигм: от вариационных автоэнкодеров к состязательному обучению, от многоэтапных методов редактирования к предобученным трансформерам и, наконец, к большим языковым моделям (LLM).

Перед рассмотрением эволюции методов, необходимо привести формальную постановку задачи и ее вариации.

Форматы обучающих данных

По характеру обучающих данных выделяются три основных вида:

- **Параллельные данные** – наличие высказываний в парах (x, x') , выражающих одно и то же содержание в разных стилях. Такие пары позволяют применять стандартное обучение по схеме seq2seq, но на практике такие корпуса крайне редки и дороги в создании.
- **Непараллельные данные** – наиболее распространенный формат, при котором у текстов имеются заданные стилевые метки, но отсутствуют

параллельные пары (эти же объекты в другом стиле). Такие данные доминируют в исследованиях TST [4].

- **Few-shot и zero-shot** – методы, при которых для описания целевого стиля используется всего несколько примеров или текстовая инструкция. Данные подходы активно применяются с момента появления LLM [5].

Используемые постановки задачи

В обзорной работе [4] предлагается следующая постановка задачи: пусть дан исходный текст x с атрибутом стиля s_0 , необходимо сгенерировать высказывание x' в целевом стиле s_1 с сохранением семантики x , то есть смоделировать распределение $p(x' \mid s_1, x)$.

Такой подход предполагает наличие представления стилевого атрибута в дискретном виде: «позитивный/негативный», «формальный/неформальный» и т.п., что соответствует задачам переноса тона или формальности. Несмотря на доминирование таких задач в области, TST не ограничивается переносом конечного числа признаков. Так, в задачах, связанных с индивидуальным слогом автора или жанровой принадлежностью, стиль определяется не бинарной меткой, а совокупностью лексических, синтаксических и дискурсивных характеристик, извлекаемых из корпуса.

Задача, рассматриваемая в настоящей работе, отличается от классической постановки TST по двум существенным параметрам. Во-первых, стиль определяется не как дискретный атрибут, а как *коллективный стиль сообщества* – совокупность характерных слов, оборотов, синтаксических конструкций, жаргонизмов, складывающихся из откликов множества авторов определенного сообщества. Во-вторых, формальная постановка на манер [4] имела бы целью смоделировать распределение

$$p(x'_a \mid y),$$

где y – текст поста, а x'_a – комментарий, неявно несущий стиливой атрибут a , усвоенный моделью в процессе дообучения. В отличие от классической формулировки, здесь стиль не задается явно на этапе генерации, а обобщается

моделью через обучение на корпусе целевого сообщества. Такая постановка ближе к задаче условной генерации текста (conditional text generation) со стиливым ограничением, чем к задаче переноса стиля в узком смысле.

Эволюция методов стилизации

Управляемая генерация с помощью VAE

Широкое применение нейросетей в TST началось с работы [6] 2017 года, в которой вариационные автоэнкодеры (VAE) были адаптированы для обработки дискретного текста. Модель обучалась совместно с дискриминаторами атрибутов, это позволило латентному пространству¹ быть разделимым на семантическое содержание и стиливую составляющую. На этапе генерации исходные компоненты стиля подменялись целевыми, изменяя тональность текста.

Примерно в то же время в работе [3] авторы предложили состязательный автоэнкодер – модель, в которой энкодер сжимает текст в латентное представление, а дискриминатор пытается определить стиль по этому представлению. В результате энкодер обучается строить общее латентное пространство для текстов обоих стилей, удаляя из представлений стиливую информацию. Стиль задаётся только на этапе декодирования через отдельный вектор. Предложенный авторами принцип перекрёстного выравнивания (по представлению текста одного стиля генерировать текст в другом) оказал заметное влияние на последующие работы.

В статье [2] предложили два практических варианта архитектуры: (а) стиливое представление, добавляемое к латентному вектору общего энкодера, и (б) декодеры, специфичные для каждого стиля. Кроме того, авторы ввели метрику сохранения содержания (content preservation), показав, что одной лишь точности переноса стиля недостаточно для полноценной оценки.

Позже в работе [7] был развит подход на основе разделения представлений путем введения многозадачных (в связи с разбиением исходной цели на подзадачи) и состязательных функций потерь, явно направленных на выде-

¹пространство $\mathbb{R}^d$, в которое энкодер отображает входные векторы; играет роль сжатого промежуточного представления между кодированием и декодированием.

ление стилевой информации в отдельное латентное подпространство. Такое разделение показало заметные результаты, подтвердив качественность метода.

К преимуществам VAE-методов относятся интерпретируемость латентного пространства и возможность управляемой интерполяции между стилями (при взятии двух латентных векторов текста со стилями s_0 и s_1 , подача точки на отрезке между ними в декодер приведет к сочетанию стилей). Основными ограничениями являются проблема коллапса апостериорного распределения (posterior collapse) и неполное разделение стилевых и содержательных признаков.

Состязательное обучение и подходы на основе GAN

Успех архитектуры CycleGAN [8] в области компьютерного зрения вдохновил исследователей на ряд работ, в которых применялись состязательные методы непосредственно к генерации последовательностей, минуя необходимость в явном разделении латентного пространства.

В статье [9] заменили традиционные дискриминаторы предварительно обученными языковыми моделями, обеспечив более богатый обучающий сигнал, одновременно гарантирующий большую грамматическую и стилистическую естественность текста, измеряемую перплексией, и точность переноса стиля.

В другой работе того же года [10] было предложено использовать машинный перевод как сохраняющую содержание проекцию: перевод исходного текста на промежуточный язык удалял стилистические маркеры, а обратный перевод с обуславливанием целевого стиля генерировал результат.

В статье [11] расширили состязательные подходы для одновременного переноса нескольких атрибутов (например, тональности и времени), сделав шаг к более общей управляемости генерацией.

Применение GAN-подходов к тексту, несмотря на все преимущества, сопряжено с архитектурными трудностями, главной из которых является недифференцируемость дискретной выборки токенов. Данный класс моделей изначально был спроектирован под задачи создания изображений, в которых

генератор выдаёт непрерывный вектор, для которого считается градиент. Но в случае выдачи моделью текста на каждом шаге выбирается один токен из словаря, тогда выходной вектор, поступающий в дискриминатор, состоит из одной единицы и нулей, и при подсчете градиента частные производные либо нулевые, либо неопределены, вследствие чего модель не обучается. Для преодоления этого ограничения использовались методы, позволяющие генератору выдавать векторы, близкие к единичным, но с непрерывным распределением величин, или сводящие процесс обучения модели к обучению с подкреплением, в котором генератор предстает агентом, выбранный токен – действием, а оценка дискриминатора – наградой.

Многоэтапные методы

Параллельно с генеративными подходами развивались методы, основанные на нахождении и замене содержащих стиль токенов. В работе [12] был предложен трехэтапный подход *Delete, Retrieve, Generate*: (1) удаление токенов, тесно связанных с исходным стилем; (2) извлечение из базы всех высказываний целевого стиля со схожим содержанием; (3) генерация результата с учетом сохраненного содержания и извлеченного шаблона. Несмотря на простоту, такой подход оказался высоко конкурентоспособным.

Предобученные трансформеры

Появление предобученных моделей – BERT [13], GPT-2 [14], BART [15], T5 [16] – существенно изменило область TST. Такие модели несут богатые знания языкового моделирования, полученные при обучении на большом корпусе, и могут эффективно дообучаться на относительно небольших стилистических наборах данных.

Так, в работе [17] был представлен TextSETTR – метод *few-shot* извлечения стиля на основе T5, при котором модель извлекает вектор стиля из нескольких примеров и использует его для изменения стиля входных предложений, обеспечивая высокие обобщающие способности на новых стилях без параллельных обучающих данных. Данный подход ознаменовал переход от специфичных архитектур к универсальным предобученным.

Переход к LLM и обоснование подхода настоящей работы

С 2022 года большие языковые модели – GPT-3/4, LLaMA, Mistral и другие – принципиально изменили подход к задачам стилизации текста. Масштабное предобучение на разнородных текстах наделяет LLM неявными стилистическими знаниями, а способность следовать инструкциям позволяет выполнять качественный перенос стиля в режимах zero-shot (модели подается исключительно инструкция) и few-shot (помимо инструкции приводится от одного до нескольких десятков размеченных примеров) без узкоспециализированного обучения [5].

На практике наиболее распространенным способом управления стилем генерации LLM является prompt-tuning – модификация входного запроса с описанием ожидаемого стиля. Как было отмечено во введении, данный подход обладает низкой стоимостью внедрения, но расходует контекстное окно, увеличивает вычислительные затраты и плохо применим к моделям с ограниченным числом параметров [1].

Важным аргументом в пользу дообучения является работа [18], в которой проведено систематическое сравнение промптинга и дообучения LLM на таких задачах TST как перенос тональности и детоксификация для трех языков: английского, хинди и бенгальского. Авторы установили, что дообучение существенно улучшает результаты по сравнению с zero-shot и few-shot промптингами, делая их сопоставимыми с предшествующими "state-of-the-art" (демонстрирующими наилучшее качество) методами. Это подчеркивает необходимость дообучения для достижения высокого качества стиливой адаптации, что обосновывает выбор методов, рассматриваемых в данной работе.

Актуальность постановки задачи

Обзор [4] и ряд последующих работ отмечают, что опора на датасет Yelp Sentiment [19] в качестве основного бенчмарка привела к переобучению методов под конкретный домен. На протяжении 2023–2025 годов в сообществе усиливаются призывы к более общим бенчмаркам – многоязычным и многодоменным. Большинство существующих исследований оперируют би-

нарными стилистическими атрибутами, тогда как реальная стилистическая вариативность значительно богаче.

Постановка задачи настоящей работы – усвоение коллективного стиля сообщества, представляющего собой совокупность характерных лексических, синтаксических и дискурсивных особенностей множества авторов, – является актуальной в контексте указанной критики.

Во-первых, стиль здесь не сводится к бинарному атрибуту, а неформально определяется обучающим корпусом. Во-вторых, используется русскоязычный датасет комментариев из Telegram, что вносит вклад в расширение языкового и доменного разнообразия исследований в области TST. В-третьих, рассматривается задача условной генерации (комментарий по посту), а не классического переноса стиля между заданными категориями, что расширяет подход к стилизации от переноса атрибутов с источника на целевой объект до генерации, имеющей стилевое соответствие и семантическую согласованность с предварительно заданным текстом.

Глава 1. Методы адаптации языковых моделей

1.1 Обзор существующих методов дообучения LLM

Полное дообучение

Современные методы обработки естественного языка (Natural Language Processing, NLP), основанные на архитектуре трансформеров, как правило, требуют значительных вычислительных ресурсов и больших объемов данных для обучения. Несмотря на то, что этап предварительного обучения языковых моделей осуществляется на неразмеченных данных, адаптация модели к конкретной задаче или домену часто предполагает использование размеченных корпусов, что может быть трудоемким и дорогостоящим.

В условиях ограниченного объема данных стандартным подходом в нейронных сетях является transfer learning – метод, при котором модель инициализируется весами, полученными при обучении на данных из иной предметной области, после чего дообучается под частную задачу на относительно небольшой специализированной выборке. В задачах NLP широко использу-

ется следующая схема: на первом этапе, обучаясь предсказанию следующего слова, языковая модель перенимает статистические и грамматические закономерности языка и обретает навыки языкового моделирования [14]. На втором – выполняется дообучение модели под конкретную область применения. Данный этап принято называть дообучение (fine-tuning) [20].

Применение fine-tuning позволяет существенно сократить требуемый объем обучающих данных, но подход все же остается вычислительно затратным, поскольку предполагает обновление всех параметров модели, которых современные реализации могут насчитывать до нескольких триллионов [21]. Кроме того, при некорректном выборе гиперпараметров полное дообучение может приводить к явлению катастрофического забывания, выражающемуся в утрате моделью ранее приобретенных обобщающих способностей и снижении устойчивости генерации [22]. Указанные ограничения послужили предпосылкой для разработки альтернативных подходов адаптации языковых моделей, ориентированных на снижение числа обучаемых параметров.

В задачах трансфера стиля, как правило, используются относительно небольшие и шумные корпуса данных, что дополнительно усиливает перечисленные ограничения полного дообучения.

PEFT-методы

Методы параметро-эффективного дообучения (Parameter-Efficient Fine-Tuning, PEFT) представляют собой класс подходов, направленных на адаптацию предварительно обученных языковых моделей при минимальном числе обновляемых параметров. Основная идея PEFT заключается в сохранении большинства весов исходной модели фиксированными и обучении лишь ограниченного набора дополнительных или модифицируемых параметров, что позволяет существенно снизить затраты на вычисление и хранение [1].

В обзоре [1] выделяются три основных класса PEFT-методов: *аддитивные*, добавляющие небольшие обучаемые модули (адаптеры) или обучаемые входные эмбединги (soft prompt-tuning); *выборочные*, обновляющие лишь часть существующих параметров модели; и *методы повторной параметризации*, аппроксимирующие обновление весов через низкоранговые матричные

представления. В настоящей работе рассматриваются метод LoRA, относящийся к третьему классу, и метод (IA)³, сочетающий черты аддитивного подхода с масштабированием внутренних активаций.

Метод LoRA

Метод LoRA (Low-Rank Adaptation) [23] направлен на снижение затрачиваемых вычислений и памяти при дообучении больших языковых моделей за счет низкоранговой аппроксимации обновлений весов. В отличие от полного дообучения, при котором оптимизируются все параметры модели, в LoRA веса исходной модели остаются неизменными, а обучаются только параметры двух низкоранговых матриц.

Пусть $W \in \mathbb{R}_{d \times l}$ – матрица весов исходной модели. В LoRA обновление параметров представляется в виде $\Delta W = A \times B$, где $A \in \mathbb{R}_{d \times r}$, $B \in \mathbb{R}_{r \times l}$, а $r \ll \min(d, l)$ – гиперпараметр ранга. Итоговые веса модели после дообучения принимают вид $W + \Delta W = W + A \times B$.

Обоснование эффективности данного подхода связано с гипотезой о существовании внутреннего подпространства (intrinsic dimensionality) в пространстве параметров нейронных сетей, согласно которой оптимальное решение задачи дообучения лежит в подпространстве существенно меньшей размерности по сравнению с полным пространством, что делает возможным обучение лишь ограниченного числа параметров в пространстве весов [24]. В той же работе отмечается, что такое многообразие тем меньше и четче выделяемо, чем больше размер модели.

На практике в трансформерах LoRA чаще всего применяется к матрицам внимания W_Q , W_K и W_V . В оригинальной работе [23] метод был протестирован на GPT-3, где число обучаемых параметров удалось сократить примерно в 5000 раз – с 175 млрд до 37.7 млн – при сохранении качества, сопоставимого с полным fine-tuning.

Важным преимуществом LoRA является возможность хранения нескольких наборов матриц A и B для различных задач при использовании одной и той же базовой модели. Это делает метод особенно удобным для задач стилизации текста. Кроме того, объединение весов исходной и дообученной матриц

является простой в вычислительном плане операцией, в связи с чем не возникает задержек при работе модели.

Метод (IA)³

Метод (IA)³ (Infused Adapter by Inhibiting and Amplifying Inner Activations, встроенный адаптер, подавляющий и усиливающий внутренние активации) представляет собой параметро-эффективный подход к дообучению LLM, основанный на масштабировании внутренних активаций вместо обновления весовых матриц. В отличие от других аддитивных методов и LoRA, (IA)³ оптимизирует лишь небольшое число векторов масштабирования, оставляя все веса модели фиксированными [25]. В стандартном трансформере вычисления в блоке внимания имеют вид:

$$Q = X \times W_Q, K = X \times W_K, V = X \times W_V$$

В методе (IA)³ вводятся обучаемые векторы l_k, l_v и l_f , которые масштабируют активации следующим образом:

$$K' = K \times \text{diag}(l_k), V' = V \times \text{diag}(l_v),$$

также изменяются выходы полносвязных слоев посредством вектора l_f .

Все векторы l_k, l_v и l_f имеют размерность d , что приводит к крайне малому числу обучаемых параметров – приблизительно 0,01–0,02% от общего числа параметров модели [25]. Вычислительная сложность метода составляет $O(d)$, что делает его более легким по сравнению с LoRA, имеющим сложность $O(r \times d)$.

Несмотря на простоту, (IA)³ демонстрирует конкурентоспособные результаты на ряде задач, особенно в условиях жестких ограничений на вычислительные ресурсы. В отличие от методов, основанных на адаптерах, (IA)³ не увеличивает время генерации, поскольку обученные векторы не изменяют вычислительную сложность.

Следует отметить, что с точки зрения задач стилизации (IA)³ и LoRA концептуально различаются: LoRA осуществляет смещение модели в про-

странстве параметров, тогда как (IA)³ регулирует вклад уже существующих активаций. Это расхождение может приводить к различным эффектам адаптации, которые предполагается исследовать в рамках данной работы.

1.2 Применимость PEFT-методов к задачам стилизации

При работе с текстом под стилем обычно понимается набор устойчивых лингвистических характеристик, не связанных напрямую с семантикой сообщения. Среди таких характеристик выделяют тональность, синтаксические конструкции, лексику, речевые шаблоны.

Как было замечено ранее, современные LLM обучаются на больших корпусах текста, обладающих значительным стилистическим разнообразием. В результате модель неявно кодирует стилистические закономерности, при этом каждому выделяемому стилевому атрибуту соответствует собственное внутреннее подпространство параметров (intrinsic dimensionality) [24]. Это позволяет рассматривать стилевую адаптацию как поиск и усиление соответствующего подмножества параметров, отвечающих за заданный стиль, что теоретически обосновывает достаточность использования параметро-эффективных методов дообучения.

Эмпирическое подтверждение данного тезиса получено в ранее упомянутой работе [18], где проведено систематическое сравнение prompt-tuning и дообучения LLM на задачах переноса тональности и детоксификации для трех языков.

Дополнительным аргументом в пользу дообучения служат ограничения промптинга, описанные в обзоре [1]: ограниченная длина контекстного окна и квадратичный рост вычислительных затрат при увеличении длины последовательностей. Указанные ограничения делают управление стилем через промпт малоприменимым для моделей с малым числом параметров, рассматриваемым в настоящей работе.

Таким образом, PEFT-методы представляются перспективным инструментом для стилевой адаптации: они позволяют учитывать специфику стилового корпуса при минимальных вычислительных затратах и, в отличие от промптинга, не затрачивают контекстное окно на описание стиля. Кроме того,

возможность хранения компактных наборов обученных параметров (матриц LoRA или векторов (IA)³) при неизменной базовой модели наделяет эти методы большей практической ценностью по сравнению с полным дообучением для сценариев, в которых одна модель должна поддерживать несколько стилевых адаптаций одновременно.

Глава 2. Данные и постановка прикладной задачи

2.1 Постановка задачи

Задача состоит в обучении языковой модели генерировать комментарии в стиле откликов в Telegram. Модель получает текст поста и по нему должна сгенерировать стилистически согласованный отклик.

Пусть задан датасет

$$\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N,$$

где $x^{(i)}$ – текст поста, $y^{(i)} = (y_1^{(i)}, \dots, y_m^{(i)})$ – последовательность токенов отклика. LLM моделирует условное распределение токенов:

$$p_{\theta}(y^{(i)} | x^{(i)}) = \prod_{j=1}^m p_{\theta}(y_j^{(i)} | x^{(i)}, y_{<j}^{(i)}).$$

Тогда функция потерь имеет вид

$$\mathcal{L}(\theta) = - \sum_{i=1}^N \sum_{j=1}^m \log p_{\theta}(y_j^{(i)} | x^{(i)}, y_{<j}^{(i)}).$$

Таким образом, задача сводится к каузальному языковому моделированию, в котором минимизируется кросс-энтропия по токенам комментария при условии текста поста. Кросс-энтропия может быть представлена в виде

$$CE(p, q) = H(p) + D_{KL}(p||q),$$

где p – истинное распределение, q – распределение модели, а $H(p)$ – энтропия данных, не зависящая от параметров модели θ , $D_{KL}(p||q)$ – дивергенция Кульбака – Лейблера распределения q относительно p . Таким образом, задача состоит в сведении генерируемого моделью распределения к распределению токенов в исходном датасете. Это равносильно описанной в главе 1 постановке в виде моделирования $p(x'_a | y)$, где стилиевой атрибут a усваивается моделью через дообучение на целевом корпусе.

Каждый обучающий пример формируется в виде единой последовательности токенов

$$X = (x_1, \dots, x_k, y_1, \dots, y_m),$$

где $x_1, \dots, x_k$ – токены поста и служебного разделителя «Пост: <текст> Комментарий:», а $y_1, \dots, y_m$ – токены комментария. Функция потерь вычисляется только по токенам комментария; токены поста используются исключительно как каузальный контекст и маскируются при вычислении оптимизируемой функции потерь.

2.2 Описание датасета

В качестве данных использовался собранный с 7 публичных Telegram-каналов датасет следующих тематик: «новости», «искусственный интеллект» и «коммуникация». Исходный корпус включает 5000 постов и к каждому из них в среднем по 3 комментария. В результате исходный объем датасета составил 14 431 комментарий.

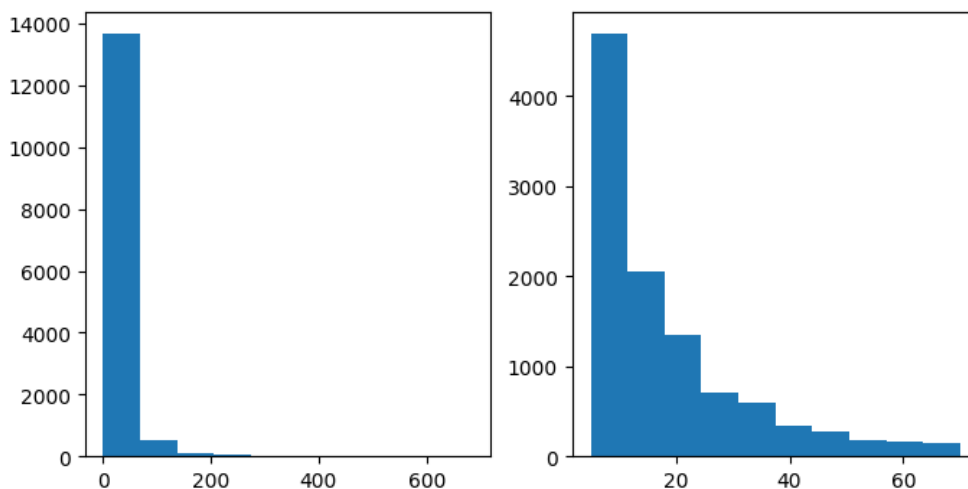


Рис. 1: Распределение количества слов в комментарии до и после очистки

Предобработка данных включала в себя удаление ссылок² и маркировок каналов, персональных данных, комментариев, содержащих меньше 5 и больше 70 слов, перевод эмодзи в текстовое представление. После этого количество уникальных постов составило 4637, всех комментариев – 10300,

²Удаление ссылок применялось только к комментариям: такой подход позволяет модели учитывать наличие ссылок во входном контексте, но не обучаться их генерировать.

средняя длина комментария – 20,5 слов. На рисунке 1 представлена гистограмма распределения длин комментариев до и после обработки. Выборка была разделена на тренировочную, валидационную и тестовую в пропорциях 80/10/10, после чего реорганизована в словарь «пост» – «комментарий».

Обработанный анонимизированный датасет опубликован на платформе HuggingFace [26].

Глава 3. Эксперименты и результаты

3.1 Методы дообучения

Рассматриваются три подхода к дообучению.

Полное дообучение (Full Fine-Tuning) – обновление всех параметров модели. Из-за полного изменения матрицы весов возможна проблема катастрофического забывания – сильных потерь способностей языкового моделирования и знаний о мире.

Low-Rank Adaptation (LoRA) [23] – дообучение малой части параметров, в ходе которого обновляются веса матриц модели посредством низкоранговой аппроксимации $\Delta W = A_{d \times r} \times B_{r \times d}$, где $r \ll d$. Применимость метода обосновывается гипотезой о том, что оптимальное решение downstream-задачи лежит в низкоразмерном подпространстве пространства параметров модели [24].

(IA)³ (Infused Adapter by Inhibiting and Amplifying Inner Activations) [25] – параметро-эффективный подход к дообучению, основанный на масштабировании внутренних активаций модели. Все исходные веса остаются фиксированными; обучаются лишь три вектора внутренней размерности d : l_k , l_v , масштабирующие keys и values в блоке внимания:

$$K' = K \cdot \text{diag}(l_k), \quad V' = V \cdot \text{diag}(l_v)$$

и l_f , изменяющие выходы полносвязных слоёв трансформера.

Концептуальное отличие от LoRA состоит в том, что LoRA смещает модель в пространстве параметров через низкоранговое обновление весов, в то время как (IA)³ усиливает или подавляет уже существующие активации.

Применительно к задаче это означает, что (IA)³ может быть особенно эффективен, когда целевой стиль уже латентно представлен в предобученной модели и требуется лишь перебалансировать вклад имеющихся признаков.

Все три метода реализуются как продолжение обучения задаче каузального языкового моделирования с функцией ошибки в виде кросс-энтропии. Токены поста маскируются и не участвуют в оптимизации.

3.2 Используемые метрики

В результате экспериментов выявлено, что функция потерь на валидационной выборке не является надежным критерием останова для задач стилизации. Функция ошибок каузального языкового моделирования оценивает вероятность конкретного образцового отклика. Однако для каждого поста существует множество допустимых комментариев. На ранних этапах модель выучивает частотные шаблоны, уменьшая потери, но прибегая к наиболее общим и семантически не связанным с источником комментариям. На поздних – генерирует более релевантные посту комментарии, не совпадающие с референсным значением, поэтому значение функции потерь растет, несмотря на фактическое улучшение качества.

По этой причине были введены дополнительные метрики на основе *Distinct- n* . Стандартная реализация была модифицирована: разбиение производится по токенам вместо слов, что лучше выявляет заикливания; также добавлен подсчет разнообразия внутри каждого отклика. По итогу метрика *combined_distinct-2* (далее *comb_d2*) имеет вид

$$\text{comb_d2} = \text{d2_tok} \times \text{self_d2}^{\beta},$$

где первый множитель отражает разнообразие между откликами, второй – внутри каждого из них, гиперпараметр β регулирует вклад второго множителя. В работе использовалось значение по умолчанию $\beta = 1$.

В ходе экспериментов метрика была доработана до *combined_distinct-2_x_len* (далее *comb_d2xlen*), включающей нормализацию по длине генера-

ции:

$$\text{comb_d2_x_len} = \text{d2_tok} \times \text{self_d2}^\beta \times \min \left(1, \frac{\text{mean_len}}{\text{norm_len}} \right),$$

где mean_len – средняя длина сгенерированных откликов для измеряемой конфигурации, а norm_len – средняя длина откликов в обучающей выборке, равная 20,5 словам. Третий множитель штрафует модели за слишком короткие генерации. Необходимость взятия минимума следует из того, что модели после предобучения склонны генерировать длинные последовательности, и за счет этого набирают высокие баллы ($\text{ruGPT-3-small} = 0,97$, $\text{Qwen2.5-0.5B} = 0,85$). Такая адаптация позволяет справедливо сравнивать малые генерации, на которых моделям легче избежать заиканий, с большими. Взятие минимума ограничивает метрику отрезком $[0,1]$, что упрощает сравнение.

Метрика разнообразия не учитывает релевантность ответа, в связи с чем дополнительно используется BERTScore [27], определяющий семантическую близость между постом или комментариями и сгенерированным текстом. При подсчете близости между постом и откликом фиксируется Precision – степень подкрепленности выданных токенов текстом источника; при сравнении исходных и сгенерированных комментариев – F1-мера, учитывающая близость в обоих направлениях.

Для оценки потери исходных знаний подсчитывалась перплексия по русскоязычному корпусу Wikipedia [28], для оценки качества адаптации (fluency) – по комментариям тестовой выборки.

3.3 Модель ruGPT3-small

Изначально моделью для экспериментов была выбрана ruGPT-3-small – русскоязычная авторегрессионная языковая модель из семейства ruGPT-3, разработанного на основе архитектуры GPT-2 [14] и опубликованного командой SberDevices [29]. Предобучение моделей проводилось на большом русскоязычном корпусе, включающем тексты из различных источников, что обеспечивает понимание русского языка и владение общими знаниями о мире. Выбор данного семейства обусловлен наличием нативной поддержки русско-

го языка на уровне токенизатора и предобучающего корпуса, что позволяет избежать потерь при кодировании кириллического текста. Семейство ruGPT-3 включает модели нескольких размеров (small (125M), medium (355M), large (760M)), из которых small была выбрана в качестве отправной точки по причине минимальных требований к вычислительным ресурсам и удобства для отладки экспериментальной процедуры.

Полное дообучение проводилось с параметрами AdamW, $lr=5 \cdot 10^{-5}$, cosine scheduler, batch size=8. Выбор конфигурации длин последовательностей в токенах производился из трех пар (128, 256), (256, 384), (384, 512), метрики отображены в таблице 1. В результате были выбраны параметры post=256, total=384.

В серии основных экспериментов по трансферу стиля модель продемонстрировала следующее поведение: вместо обобщения задачи она кластеризовывала посты по эмоциональному содержанию и запоминала шаблонные комментарии для каждого кластера. Это выразилось в низком разнообразии (comb_d2 = 0,473) и высокой перплексии на комментариях (77,67 на эпохе 3). Перплексия на Wikipedia выросла с 22,60 до 51,54, что свидетельствует о катастрофическом забывании. Рост ошибки на валидационной выборке сигнализирует о переобучении. В связи с этим была выдвинута гипотеза о том, что из-за малого числа параметров подмножество весов, соответствующих данной задаче, имеет слишком высокую внутреннюю размерность [24] или вовсе не выражено, из-за чего модель не смогла достаточно обобщить тренировочный датасет. Чтобы проверить, существует ли связь между размером и качеством обобщения в этой задаче, в той же конфигурации был проведен эксперимент с medium размером модели на 355M параметров, результаты которого представлены в таблице 1.

Medium конфигурация ruGPT-3 продемонстрировала заметный прирост в качестве. Но значение функции потерь на валидационной выборке все же не падало, а ruGPT-3-large в форматах хранения весов FP-32 и FP-16 не удовлетворял имеющимся ограничениям на вычислительные ресурсы. По этой причине дальнейшие эксперименты было решено проводить на модели Qwen2.5-0.5B. Следует отметить, что ruGPT3 и Qwen различаются не только числом параметров, но и архитектурой, токенизатором и данными предобучения, что

Таблица 1: Сравнение конфигураций ruGPT-3

Параметры	Val loss эп.1	Val loss эп.2	Val loss эп.3	Distinct- 2 эп.1	Distinct- 2 эп.2	Distinct- 2 эп.3	Время обуче- ния
post=128, total=192	4,329	4,410	4,483	0,371	0,554	0,680	17 мин
post=256, total=384	4,275	4,340	4,403	0,511	0,588	0,650	29 мин
post=384, total=512	4,269	4,332	4,391	0,456	0,570	0,674	36 мин
ruGPT-3- medium (256, 384)	3,840	3,874	3,940	0,749	0,794	0,760	90 мин

не позволяет изолировать влияние отдельных факторов.

3.4 Модель Qwen2.5-0.5B

Qwen2.5-0.5B – мультиязычная авторегрессионная языковая модель из семейства Qwen2.5 [30], содержащая 494М параметров. В отличие от ruGPT3, модель предобучена на крупном многоязычном корпусе и имеет отличия в архитектуре (Grouped Query Attention (GQA), RMSNorm и SwiGLU-активации), которые, впрочем, не являются критическими в рассматриваемой задаче.

Анализ покрытия токенизатора показал, что Qwen обрабатывает кириллический текст менее эффективно, чем ruGPT3: в 256 токенов помещается лишь 55% постов против 68% у ruGPT3. В связи с этим для Qwen выбрана конфигурация post=384, total=512, обеспечивающая покрытие $\approx 71\%$ постов. Остальные параметры идентичны.

Полное дообучение.

При полном дообучении модель продемонстрировала быструю адаптацию к домену: перплексия на комментариях снизилась до 38,19 уже ко второй эпохе. Однако на следующей эпохе значение функции потерь на тренировочной выборке упало до 1,16 при резком росте потерь на валидационной (3,63 $\rightarrow$ 4,30), что свидетельствует о запоминании обучающих данных. Парал-

тельно перплексия на Wikipedia возросла с 11,28 до 105,44 к третьей эпохе. Качественный анализ показал, что на ранних эпохах генерация вырождена (зацикливание токенов); на эпохе 2 комментарии наиболее связаны с постом, а на эпохе 3 появляются артефакты генерации. При этом метрики разнообразия растут во все три эпохи, но с разной скоростью, из чего следует, что они являются необходимыми, но недостаточными критериями для этой задачи. По этой причине метрика `combined_d2` была доработана в `combined_d2xlen` за счет добавления нового множителя – отношения средней длины сгенерированного текста на среднюю длину откликов в обучающей выборке. Динамика функции потерь представлена на рисунке 2.

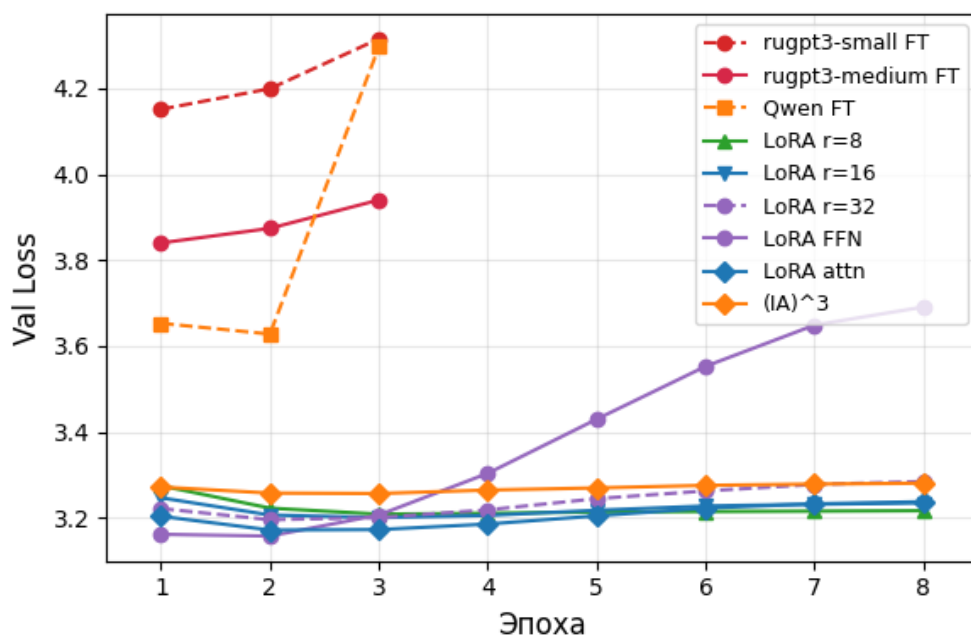


Рис. 2: Динамика функции потерь на валидационной выборке

Метод LoRA.

Изначально были протестированы три конфигурации: $r=8$ ($\alpha=16$), $r=16$ ($\alpha=32$) и $r=32$ ($\alpha=64$) с target modules – q_proj и v_proj ; шагом обучения $lr=2 \cdot 10^{-4}$; 8 эпох. Выбор гиперпараметров обосновывался их использованием в оригинальной статье [23]. Адаптация к домену оказалась сильнее, чем при полном дообучении: перплексия на комментариях составила 24,53–25,88 на лучших эпохах против 38,19 у полного дообучения. Забывание значительно мягче – перплексия на Wikipedia для лучших чекпоинтов LoRA составляет

22,48–36,92 против 105,44, визуализация представлена на рисунке 3. Функция потерь на валидационной выборке изменяется плавно ($3,22 \rightarrow 3,28$ за 8 эпох при $r=32$) в отличие от резкого скачка при полном дообучении. Лексическое разнообразие стабильно высокое с первой эпохи ($\text{comb_d2} \approx 0,72\text{--}0,78$), comb_d2xlen колеблется от 0,36 до 0,51.

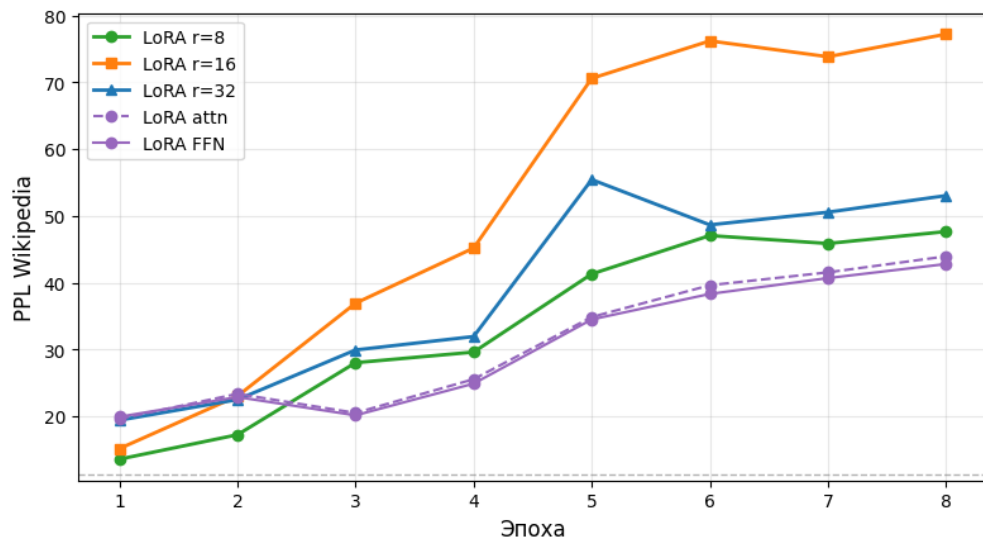


Рис. 3: Динамика забывания при дообучении LoRA

Различия между тремя конфигурациями LoRA по метрикам адаптации минимальны: prpl_comments варьируется от 24,75 ($r=32$) до 25,16 ($r=8$). Примечательно, что предполагаемая связь роста степени забывания с увеличением числа обучаемых параметров нарушается LoRA $r=16$: к эпохе 8 prpl_wiki составляет 49,30 ($r=8$) и 58,52 ($r=32$), а для $r=16$ достигает 78,12. Оптимальной точкой по балансу адаптации и сохранения знаний являются ранние эпохи (2–3), сравнение также представлено на рисунке 4

Качественный анализ генераций выявил, что LoRA сохраняет способность генерировать грамматически связный текст заметно длиннее (количество слов в генерациях на тестовой выборке на лучших эпохах колеблется от 13,52 до 14,99), чем полное дообучение (на второй эпохе результат 12.55 при более низком лексическом разнообразии). При этом BERTScore Precision между генерацией и постом для LoRA (0,58–0,6) уступает значению базовой модели (0,632), что говорит о том, что LoRA в первую очередь адаптирует стилистику, а не семантическую связь. Следует учитывать, что LoRA генерирует более длинные связные комментарии, а BERTScore Precision чувствителен к

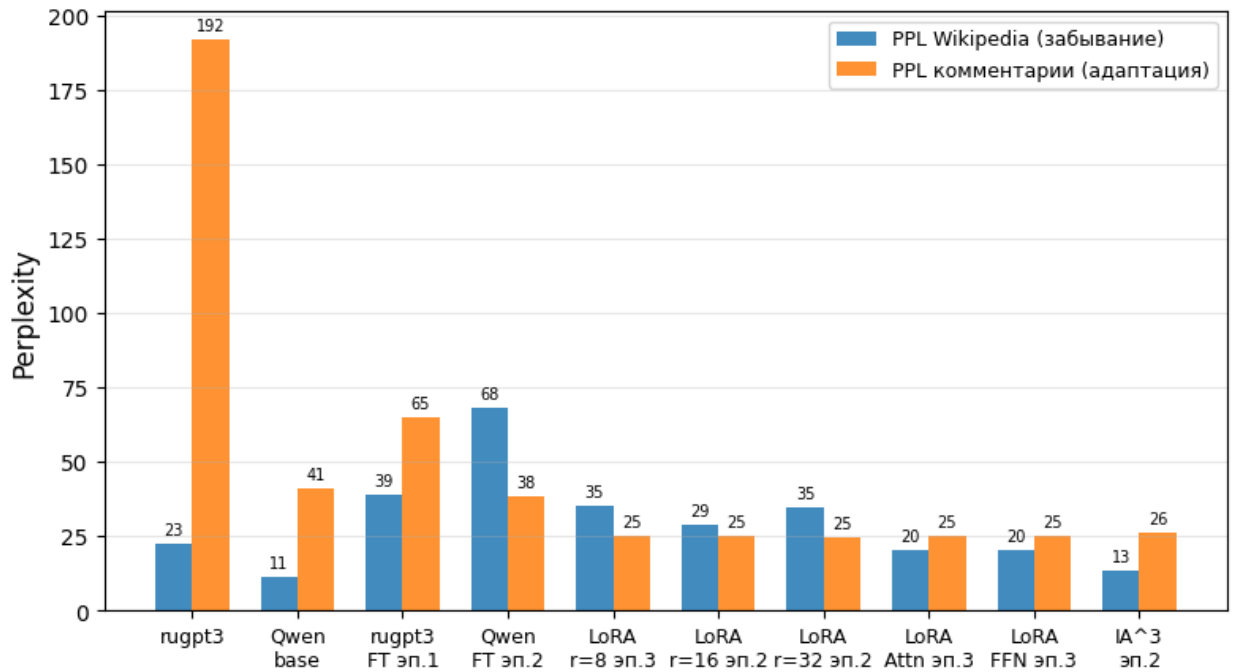


Рис. 4: Перплексия: адаптация к домену vs катастрофическое забывание

длине генерации, что ограничивает прямую сопоставимость значений между методами.

Таблица 2: Динамика валидационных метрик по эпохам

Эп.	ruGPT3 FT	Qwen FT	LoRA $r=8$	LoRA $r=16$	LoRA $r=32$	LoRA FFN	LoRA Attn	(IA) ³
Функция потерь (вал.)								
1	4,15	3,65	3,33	3,30	3,29	3,16	3,2	3,27
2	4,20	3,63	3,29	3,27	3,27	3,15	3,17	3,3
3	4,32	4,3	3,28	3,28	3,28	3,21	3,17	3,26
4	—	—	3,28	3,29	3,31	3,3	3,18	3,26
8	—	—	3,3	3,34	3,42	3,65	3,24	3,28
Combined Distinct-2								
1	0,32	0,19	0,76	0,748	0,73	0,69	0,73	0,74
2	0,46	0,43	0,74	0,747	0,77	0,72	0,69	0,77
3	0,47	0,62	0,71	0,726	0,75	0,75	0,71	0,75
4	—	—	0,73	0,76	0,75	0,75	0,7	0,75
8	—	—	0,72	0,76	0,75	0,8	0,72	0,74

Как было замечено ранее, преимущество LoRA для рассматриваемой задачи заключается в том, что метод может применяться к любым линейным слоям трансформера, включая полносвязные (FFN). В работе [31] было показано, что FFN-слои составляют около двух третьих параметров трансформера и функционируют как ассоциативная память типа «ключ–значение», в

Таблица 3: Сравнение методов дообучения (отобранные чекпоинты)

Метод	comb d2	ppl comm	ppl wiki	BS post	BS ref
ruGPT3 base	0,548	191,7	22,6	0,611	0,591
Qwen base	0,675	40,9	11,3	0,632	0,600
ruGPT3 FT (эп.1)	0,700	64,9	38,8	0,645	0,642
Qwen FT (эп.2)	0,659	38,2	67,9	0,646	0,629
LoRA $r=8$ (эп.3)	0,733	25,2	35,2	0,628	0,631
LoRA $r=16$ (эп.2)	0,745	24,9	28,8	0,621	0,626
LoRA $r=32$ (эп.2)	0,741	24,8	34,7	0,626	0,631
LoRA FFN (эп.2)	0,705	23,77	22,87	0,637	0,619
LoRA Attn (эп.2)	0,707	23,75	23,35	0,639	0,620
(IA) ³ (эп.2)	0,752	26,26	13,27	0,637	0,609

которой первые кодируют текстовые паттерны обучающих данных, а значения формируют распределения над выходным словарём. При этом нижние слои фиксируют грамматические закономерности, тогда как верхние – семантические. Это позволяет предположить, что стилевые характеристики – устойчивые лексические и синтаксические паттерны – могут храниться преимущественно в FFN-слоях. Соответственно, применение LoRA к весам FFN-блоков, может точнее воздействовать на подпространство параметров, отвечающее за стилевые особенности генерации. Для проверки данной гипотезы были проведены эксперименты с применением метода к полносвязным target modules – gate_proj, up_proj, down_proj и ко всем матрицам внимания target modules – q_proj, v_proj, k_proj и o_proj – при сохранении всех остальных гиперпараметров.

Конфигурация с применением LoRA к полносвязным слоям имеет высокое лексическое разнообразие – comb_d2 достигает 0,76 на четвертой и почти 0,79 на последних, comb_d2xlen варьируется от 0,43 до 0,56, средняя длина генерации на второй эпохе превысила 16 слов. Значения функции ошибок на валидационной выборке после первой и второй эпох практически не изменялись (3,162 и 3,158 соответственно), а затем монотонно росли до 3,7 на последней эпохе. Более быстрое достижение минимума и следующее за ним переобучение связано с большим количеством обучающих параметров. Несмотря на влияние дообучения на полносвязные веса, в которых хранятся

основные знания модели о мире, показатели ее катастрофического забывания – от 19,9 до 42,8 – не превышают ppl_wiki предыдущих экспериментов. Перплексия на комментариях растет в течение восьми эпох с 23,69 до 41,14. Подробные результаты доступны в таблице 3.5 по псевдониму LoRA FFN.

Применение метода ко всем блокам внимания демонстрирует высокие показатели: средняя длина генерации достигает 17,4 слов на 7-й эпохе, comb_d2 доходит до 0,8 на эпохе 8, а метрика лексического разнообразия с учетом длины на эпохах 6-8 сильно превышает максимум предыдущих экспериментов (0,66 на седьмой эпохе против 0,56 у LoRA FFN). Адаптация к домену численно близка к результатам предыдущего эксперимента – монотонно возрастает с 23,7 до 41,14. Перплексия на корпусе Wikipedia также незначительно отличается от результатов LoRA FFN и также имеет снижение на третьей эпохе до 20,49. Семантические метрики в обоих экспериментах, также как и в предыдущих, не имеют достаточной выразительности для проведения на их основе сравнения. В таблицах и графиках результаты представлены под псевдонимом LoRA Attn.

Метод (IA)³

Для (IA)³ были выбраны модули k_proj, v_proj, и down_proj. Ввиду того что метод воздействует на значительно меньшее количество параметров, чем LoRA, шаг обучения был выбран $lr=3 \cdot 10^{-3}$. Средняя длина генерации на второй эпохе составила 13,8, comb_d2 оказалась выше, чем у метода LoRA – 0,75 на второй эпохе и 0,77 на пятой, но comb_d2xlen осталась на уровне LoRA $r=32$ – максимальное значение 0,5 достигнуто на второй эпохе. Семантические метрики не отличаются значимо от предыдущих экспериментов. Функция ошибок на валидационной выборке колеблется от 3,257 на третьей эпохе до 3,28 на последней. Примечательно, что даже на 8 эпохе перплексия Wikipedia не превышает 15,0 – метод практически не ухудшает исходные навыки языкового моделирования модели, при этом перплексия на комментариях выше, чем у других параметро-эффективных методов, всего на несколько единиц – минимальное значение 26,26 достигается на второй эпохе.

3.5 Оценивание через LLM-as-a-Judge

Рассмотренные ранее метрики – перплексия, comb_d2, comb_d2xlen и BERTScore – оценивают отдельные аспекты генерации: адаптацию к домену, лексическое разнообразие и семантическую близость соответственно. Но ни одна из них не оценивает цельно стилистическую неотличимость сгенерированного отклика от написанного человеком. Для решения этой задачи предлагается метод оценки, основанный на подходе LLM-as-a-Judge, развивающий идеи А. Тьюринга [32].

Оценивание строится на следующей гипотезе: модель успешно усвоила коллективный стиль сообщества, если её генерации неотличимы от реальных комментариев в контексте конкретного поста. Или формально: внешний судья, которому предъявлены k реальных комментариев и один сгенерированный, не способен выявить сгенерированный с точностью, статистически превышающей случайное угадывание. Для каждого поста $x^{(i)}$ из тестовой выборки формируется набор из k_i реальных комментариев $(c_1, \dots, c_{k_i})$, $k_i \in \{1, 2, 3\}$, и одного сгенерированного $\hat{y}^{(i)}$, который ставится на случайную позицию. Судья получает текст поста и $k_i + 1$ комментариев; его задача – указать номер сгенерированного. Вероятность случайного угадывания для примера с $k_i + 1$ комментариями составляет $\frac{1}{k_i + 1}$. Пусть $a_i \in \{0, 1\}$ – индикатор верной идентификации на i -м примере. Точность судьи в эксперименте определяется как

$$\text{accuracy} = \frac{1}{N} \sum_{i=1}^N a_i,$$

а усреднённое значение случайного угадывания как

$$\text{BASE} = \frac{1}{N} \sum_{i=1}^N \frac{1}{k_i + 1},$$

равное 0,296 для сформированной подвыборки. Нулевая гипотеза H_0 : судья не способен отличить сгенерированные комментарии от реальных, то есть $\text{accuracy} = \text{BASE}$. Альтернативная гипотеза H_1 : $\text{accuracy} > \text{BASE}$. 95%-й

доверительный интервал для ассюрасу вычисляется по формуле Вальда:

$$CI = \text{accuracy} \pm 1,96 \times \sqrt{\frac{\text{accuracy} \times (1 - \text{accuracy})}{N}}$$

Описание эксперимента. В качестве модели-судьи используется GPT-5 nano, вызываемая через API OpenRouter. Выбор LLM в роли судьи, а не человека, обусловлен масштабируемостью: по итогу 8 экспериментов предстоит оценить генерации в сумме за 54 эпохи, а также ответы исходных моделей. В каждой эпохе генерировались отклики на $N = 94$ поста из тестовой выборки. Тогда придется разметить 5264 отклика. Разметка человеком такого количества данных труднореализуема. При этом оценку LLM-судьи можно рассматривать как верхнюю оценку различимости: языковая модель более чувствительна к распределению токенов и субпаттернам, таким как частотность n-грамм и перплексия текста. Используемый при оценивании промпт представлен в приложении 3.5.

Интерпретация результатов. Разность $\Delta = \text{ассюрасу} - \text{BASE}$ характеризует степень различимости: значения, близкие к нулю или отрицательные, свидетельствуют о том, что генерации не отличимы от реальных комментариев; большие значения указывают на то, что судья часто выявляет сгенерированный текст. Данная метрика дополняет ранее рассмотренные: если конфигурация эксперимента демонстрирует высокое лексическое разнообразие и низкую перплексию на комментариях, но при этом высокую Δ , то это говорит о том, что модель, несмотря на высокое качество генерации и адаптации, не смогла перенять заметные стилистические особенности сообщества.

Результаты. Итоги эксперимента представлены в приложении 3.5. Модель ruGPT3-small показала наилучшие результаты: ассюрасу составила 0,26 и 0,31 для эпох 3 и 1 соответственно, что ниже порога случайного угадывания (0,296) или незначительно превышает его. Это означает, что судья не смог выявить сгенерированные комментарии, и подтверждает наблюдения о ненадёжности значения функции ошибок на валидации как ключевой метрики в рассматриваемой задаче: несмотря на высокую перплексию и низкое лексическое разнообразие, ruGPT3-small порождала стилистически наиболее неотличимые от человеческих отклики. На следующих местах в таблице рас-

положились 4–5 эпохи (IA)³ и LoRA $r=32$. Примечательно, что конфигурации с применением LoRA ко всем блокам внимания или полносвязным слоям, несмотря на лучшие показатели по метрикам лексического разнообразия и перплексии, расположились преимущественно во второй половине таблицы (accuracy 0,49 – 0,55). Исходные модели (ruGPT3_base и qwen_base) заняли последние места – accuracy базовой Qwen составила 0,7, ruGPT3 – 0,58%. Это подтверждает эффективность проведённого дообучения: все рассмотренные методы существенно снижают различимость генераций по сравнению с исходными моделями. Также это положительно характеризует методологию метрики – исходные модели генерировали грамотный и семантически релевантный отклик, значит, они были выявлены за счет стилового несоответствия комментариям, написанным людьми.

Выводы

В ходе работы был получен ряд результатов. Подготовлен и опубликован датасет русскоязычных откликов из Telegram, разработан набор метрик для контроля хода экспериментов, предложена методология оценивания стилистической согласованности на основе подхода LLM-as-a-Judge, проведены эксперименты с методами параметро-эффективного и полного дообучения в нескольких конфигурациях.

Одним из ключевых результатов стало выявление ненадёжности функции ошибки на валидационной выборке в качестве критерия останова для задач стилизации. Кросс-энтропия по единственному референсному комментарию не отражает всё множество допустимых откликов на пост, вследствие чего стандартная процедура ранней остановки оказывается неприменимой. Этот вывод подтверждён результатами LLM-судейства: конфигурации с наименьшим значением функции потерь на валидации не показали наилучшей стилистической неотличимости.

Метрики лексического разнообразия зарекомендовали себя как необходимое, но недостаточное средство контроля: они позволяют отслеживать деградацию генерации – заикливания, шаблонные ответы, потерю грамматической связности, – однако не оценивают стилистическую и семантическую согласованность. Метрики на основе BERTScore оказались невыразительными на используемом наборе данных. Таким образом, единственной метрикой, позволившей ранжировать конфигурации на основе стилевой согласованности, оказалась оценка LLM-судьи. Это подчеркивает принципиальную сложность формализации стилевого соответствия.

Сравнительный анализ методов дообучения выявил, что применение LoRA к модулями q_{proj} и v_{proj} , а также (IA)³ обеспечивают лучшую стилистическую неотличимость по оценке LLM-судьи, чем конфигурации с большим числом обучаемых параметров – LoRA на полносвязных слоях или на всех блоках внимания. Увеличение числа обучаемых параметров улучшает метрики разнообразия и перплексии, но вносит в генерацию артефакты, распознаваемые судьёй. Таким образом, можно заключить, что стилевая адаптация эффективнее осуществляется минимальным воздействием на параметры,

а не масштабным дообучением.

Несмотря на достигнутые результаты, работа имеет ряд ограничений. Оценка стилистической согласованности опирается исключительно на LLM-судейство, которое не было провалидировано на подвыборке с человеческой разметкой, что снижает верифицируемость результатов. Эксперименты проведены на моделях малого размера, и обобщаемость выводов, в виду имеющихся ограничений объема вычислительных мощностей, на более крупные модели не проверена. Кроме того, не найдена автоматическая метрика, которая позволяла бы контролировать достаточность обучения без привлечения внешнего разметчика – её разработка, через модификацию функции ошибки либо через оценку генераций на валидационной выборке, остаётся открытой задачей.

Заключение

В работе исследована применимость методов параметро-эффективного дообучения к задаче стилевой адаптации языковых моделей на русскоязычных данных из Telegram. Подготовлен и опубликован датасет русскоязычных откликов из Telegram, а также предложена методология оценивания стилистической согласованности генераций. Показано, что методы с минимальным числом обучаемых параметров – LoRA на матрицы запросов и значений, а также (IA)³ – обеспечивают лучшую стилистическую неотличимость генераций, чем конфигурации с большим числом параметров. Установлено, что стандартные метрики – функция потерь на валидации – не позволяют надёжно оценивать качество стилизации, BERTScore не обладает выразительностью на рассматриваемом наборе данных, а лексическое разнообразие фиксируют деградацию генерации, но не учитывают стилевую и семантическую согласованность. Единственным информативным критерием стилевого соответствия оказалась оценка на основе LLM-судейства.

Список литературы

- [1] Lialin, V. Scaling down to scale up: a guide to parameter-efficient fine-tuning [Электронный ресурс] / V. Lialin, V. Deshpande, A. Rumshisky. – 2023. – URL: <https://arxiv.org/abs/2303.15647> (дата обращения: 20.03.2026).
- [2] Fu, Z. Style transfer in text: Exploration and evaluation / Z. Fu, X. Tan, N. Peng [и др.] // Proceedings of the AAAI Conference on Artificial Intelligence. – 2018. – Vol. 32.
- [3] Shen, T. Style transfer from non-parallel text by cross-alignment / T. Shen, T. Lei, R. Barzilay, T. Jaakkola // Advances in Neural Information Processing Systems (NeurIPS). – 2017. – P. 6831–6842.
- [4] Jin, D. Deep learning for text style transfer: a survey / D. Jin, Z. Jin, Z. Hu [и др.] // Computational Linguistics. – 2022. – Vol. 48, No. 1. – P. 155–205.
- [5] Reif, E. A recipe for arbitrary text style transfer with large language models / E. Reif, D. Ippolito, A. Yuan [и др.] // Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL), Short Papers. – 2022. – P. 837–848.
- [6] Hu, Z. Toward controlled generation of text / Z. Hu, Z. Yang, X. Liang [и др.] // Proceedings of the 34th International Conference on Machine Learning (ICML). – 2017. – P. 1587–1596.
- [7] John, V. Disentangled representation learning for non-parallel text style transfer / V. John, L. Mou, H. Bahuleyan, O. Vechtomova // Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL). – 2019. – P. 424–434.
- [8] Zhu, J.-Y. Unpaired image-to-image translation using cycle-consistent adversarial networks / J.-Y. Zhu, T. Park, P. Isola, A. A. Efros // Proceedings of the IEEE International Conference on Computer Vision (ICCV). – 2017. – P. 2223–2232.

- [9] Yang, Z. Unsupervised text style transfer using language models as discriminators / Z. Yang, Z. Hu, C. Dyer [и др.] // Advances in Neural Information Processing Systems (NeurIPS). – 2018. – P. 7298–7309.
- [10] Prabhumoye, S. Style transfer through back-translation / S. Prabhumoye, Y. Tsvetkov, R. Salakhutdinov, A. W. Black // Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL). – 2018. – P. 866–876.
- [11] Lample, G. Multiple-attribute text rewriting / G. Lample, S. Subramanian, E. M. Smith [и др.] // Proceedings of the 7th International Conference on Learning Representations (ICLR). – 2019.
- [12] Li, J. Delete, retrieve, generate: a simple approach to sentiment and style transfer / J. Li, R. Jia, H. He, P. Liang // Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL). – 2018. – P. 1865–1874.
- [13] Devlin, J. BERT: pre-training of deep bidirectional transformers for language understanding / J. Devlin, M.-W. Chang, K. Lee, K. Toutanova // Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL). – 2019. – P. 4171–4186.
- [14] Radford, A. Language models are unsupervised multitask learners / A. Radford, J. Wu, R. Child [и др.] // OpenAI Blog. – 2019. – Vol. 1, No. 8.
- [15] Lewis, M. BART: denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension / M. Lewis, Y. Liu, N. Goyal [и др.] // Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL). – 2020. – P. 7871–7880.
- [16] Raffel, C. Exploring the limits of transfer learning with a unified text-to-text transformer / C. Raffel, N. Shazeer, A. Roberts [и др.] // Journal of Machine Learning Research. – 2020. – Vol. 21, No. 140. – P. 1–67.

- [17] Riley, P. TextSETTR: few-shot text style extraction and tunable targeted restyling / P. Riley, N. Constant, M. Guo [и др.] // Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL). – 2021. – P. 3786–3800.
- [18] Mukherjee, S. Are large language models actually good at text style transfer? / S. Mukherjee, A. Kr. Ojha, O. Dušek // Proceedings of the 17th International Natural Language Generation Conference (INLG). – 2024. – P. 523–539.
- [19] Kaur, H. Yelp review sentiment analysis model [Электронный ресурс] / H. Kaur. – URL: <https://huggingface.co/harmanpreet-kaur/yelp-review-sentiment-analysis-model-1> (дата обращения: 20.05.2026).
- [20] Jiang, H. SMART: Robust and efficient fine-tuning for pre-trained natural language models through principled regularized optimization [Электронный ресурс] / H. Jiang, P. He, W. Chen [и др.]. – 2019. – URL: <https://arxiv.org/abs/1911.03437> (дата обращения: 20.03.2026).
- [21] Li, B. Incompressible knowledge probes: estimating black-box LLM parameter counts via factual capacity [Электронный ресурс] / B. Li. – 2026. – URL: <https://arxiv.org/abs/2604.24827> (дата обращения: 20.05.2026).
- [22] Luo, Y. An empirical study of catastrophic forgetting in large language models during continual fine-tuning [Электронный ресурс] / Y. Luo, Z. Yang, F. Meng [и др.]. – 2023. – URL: <https://arxiv.org/abs/2308.08747> (дата обращения: 20.03.2026).
- [23] Hu, E. J. LoRA: low-rank adaptation of large language models / E. J. Hu, Y. Shen, P. Wallis [и др.] // Proceedings of the 10th International Conference on Learning Representations (ICLR). – 2022.
- [24] Aghajanyan, A. Intrinsic dimensionality explains the effectiveness of language model fine-tuning [Электронный ресурс] / A. Aghajanyan, L. Zettlemoyer, S. Gupta. – 2020. – URL: <https://arxiv.org/abs/2012.13255> (дата обращения: 20.03.2026).

- [25] Liu, H. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning [Электронный ресурс] / H. Liu, D. Tam, M. Muqeeth [и др.]. – 2022. – URL: <https://arxiv.org/abs/2205.05638> (дата обращения: 20.03.2026).
- [26] Telegram post-comment pairs for style-adaptive text generation [Электронный ресурс]. – URL: https://huggingface.co/datasets/r9zhenka/telegram-post-comments_data (дата обращения: 20.03.2026).
- [27] Zhang, T. BERTScore: evaluating text generation with BERT [Электронный ресурс] / T. Zhang, V. Kishore, F. Wu [и др.]. – 2020. – URL: <https://arxiv.org/abs/1904.09675> (дата обращения: 20.03.2026).
- [28] Wikimedia downloads [Электронный ресурс] / Wikimedia Foundation. – URL: <https://dumps.wikimedia.org> (дата обращения: 20.03.2026).
- [29] Zmitrovich, D. A family of pretrained transformer language models for Russian / D. Zmitrovich, A. Abramov, A. Kalmykov [и др.] // Proceedings of LREC-COLING 2024. – P. 507–524.
- [30] Yang, A. Qwen2.5 technical report [Электронный ресурс] / A. Yang, B. Yang, B. Zhang [и др.]. – 2024. – URL: <https://arxiv.org/abs/2412.15115> (дата обращения: 20.03.2026).
- [31] Geva, M. Transformer feed-forward layers are key-value memories / M. Geva, R. Schuster, J. Berant, O. Levy // Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP). – 2021. – P. 5765–5773.
- [32] Turing, A. M. Computing machinery and intelligence / A. M. Turing // Mind. – 1950. – Vol. 59, No. 236. – P. 433–460.

Приложение

Результаты LLM-as-a-Judge

Конфигурация	Accuracy	Δ	95% CI
ruGPT3_ft_ep3	0,260	−0,039	[0,173–0,348]
ruGPT3_ft_ep1	0,312	+0,013	[0,220–0,405]
qwen_ia3_ep5	0,368	+0,069	[0,271–0,465]
qwen_lora_r32_ep4	0,379	+0,080	[0,281–0,477]
qwen_ia3_ep4	0,389	+0,090	[0,291–0,488]
qwen_lora_r16_ep8	0,400	+0,101	[0,301–0,499]
qwen_lora_r32_ep7	0,400	+0,101	[0,301–0,499]
qwen_ia3_ep3	0,411	+0,111	[0,312–0,509]
qwen_ia3_ep7	0,421	+0,122	[0,322–0,520]
qwen_lora_r16_ep6	0,421	+0,122	[0,322–0,520]
qwen_ia3_ep6	0,432	+0,132	[0,332–0,531]
qwen_lora_r32_ep5	0,432	+0,132	[0,332–0,531]
qwen_lora_r8_ep6	0,436	+0,137	[0,336–0,536]
qwen_lora_fan_ep2	0,453	+0,154	[0,353–0,553]
qwen_lora_r32_ep8	0,453	+0,154	[0,353–0,553]
qwen_ia3_ep8	0,463	+0,164	[0,363–0,563]
qwen_lora_ffn_ep3	0,463	+0,164	[0,363–0,563]
qwen_lora_r16_ep5	0,463	+0,164	[0,363–0,563]
qwen_lora_r32_ep6	0,463	+0,164	[0,363–0,563]
qwen_lora_r8_ep8	0,463	+0,164	[0,363–0,563]
qwen_lora_r8_ep7	0,468	+0,169	[0,367–0,569]
qwen_lora_r16_ep7	0,474	+0,175	[0,373–0,574]
qwen_lora_r16_ep2	0,484	+0,185	[0,382–0,585]
qwen_lora_r16_ep3	0,484	+0,185	[0,384–0,585]
qwen_lora_r16_ep4	0,484	+0,185	[0,384–0,585]
qwen_ia3_ep2	0,495	+0,195	[0,393–0,596]
qwen_lora_r32_ep2	0,495	+0,195	[0,393–0,596]

Конфигурация	Accuracy	Δ	95% CI
qwen_lora_r8_ep3	0,495	+0, 195	[0,393–0,596]
qwen_lora_r8_ep4	0,495	+0, 195	[0,393–0,596]
qwen_lora_fan_ep4	0,495	+0, 196	[0,394–0,595]
qwen_lora_fan_ep5	0,495	+0, 196	[0,394–0,595]
qwen_lora_ffn_ep4	0,495	+0, 196	[0,394–0,595]
qwen_lora_r8_ep1	0,500	+0, 200	[0,399–0,601]
qwen_lora_fan_ep3	0,505	+0, 206	[0,405–0,606]
qwen_lora_fan_ep8	0,505	+0, 206	[0,405–0,606]
qwen_lora_ffn_ep2	0,505	+0, 206	[0,405–0,606]
qwen_lora_ffn_ep7	0,505	+0, 206	[0,405–0,606]
qwen_lora_r8_ep5	0,505	+0, 206	[0,405–0,606]
qwen_ft_ep2	0,516	+0, 217	[0,415–0,616]
qwen_lora_fan_ep7	0,516	+0, 217	[0,415–0,616]
qwen_lora_r16_ep1	0,521	+0, 222	[0,420–0,622]
qwen_lora_ffn_ep5	0,526	+0, 227	[0,426–0,627]
qwen_lora_ffn_ep8	0,526	+0, 227	[0,426–0,627]
qwen_lora_r32_ep3	0,526	+0, 227	[0,426–0,627]
qwen_ft_ep3	0,537	+0, 238	[0,437–0,637]
qwen_lora_fan_ep6	0,537	+0, 238	[0,437–0,637]
qwen_lora_ffn_ep1	0,537	+0, 238	[0,437–0,637]
qwen_lora_r8_ep2	0,538	+0, 238	[0,436–0,641]
qwen_lora_fan_ep1	0,543	+0, 243	[0,442–0,643]
qwen_lora_ffn_ep6	0,547	+0, 248	[0,447–0,647]
ruGPT3_base	0,576	+0, 274	[0,475–0,677]
qwen_ia3_ep1	0,576	+0, 277	[0,475–0,677]
qwen_lora_r32_ep1	0,587	+0, 288	[0,486–0,688]
qwen_base	0,703	+0, 401	[0,609–0,797]

Используемый промпт

You are an expert evaluator of Russian-language Telegram comments.

You will be shown a Telegram post followed by several comments (from 2 to

4). Exactly ONE comment was generated by an AI model; the rest are real human comments from the channel's community. All the emojis were presented in text form.

Your task: identify which comment is AI-generated.

Respond ONLY with a valid JSON object (no markdown, no extra text):

```
{"answer": <int, 1-indexed comment number>, "confidence": <float 0.0-1.0>, "reasoning": «brief explanation in Russian, 1-2 sentences>"}
```

Примеры генерации

Post 10 (id 104)

Мощный пожар в ангаре на Маршала Новикова — у производственного цеха спецавтоматики небо заволочло густым чёрным дымом. Информация о пострадавших не поступала. Службы выехали на место.

эмоджи_лицо_в_темных_очках_

Реальные комментарии

- Ну йопрст. Понедельник же. Ну
- ну что, все уже сняли и скинули видео?

Сгенерированные

Модель	Комментарий	Детекция
rugpt3_base	Это был не тот пожар, который случился в ангаре, а другой. Какими словами можно объяснить это? Бред какой-то. Это просто не могло быть правдой. Вполне возможно, что этот пожар произошёл из-за того, что кто-то из работников навалился на рабочих, . . .	detected

Модель	Комментарий	Детекция
qwen_base	0 На месте пожара в ангаре на Маршала Новикова, у производственного цеха спецавтоматики небо заволело густым чёрным дымом. Информация о пострадавших не поступала	detected
rugpt3_ft_ep1	Такое ощущение, что горит кто-то со стороны Ангары, а люди с противоположной стороны — где-то в районе Марсово поля	NOT detected
rugpt3_ft_ep3	Такое ощущение, что кто-то просто хочет отравиться и напугать людей	NOT detected
qwen_ft_ep2	Не забывай, что я тебе скинула эмодзи_светится_от_счастья_	detected
qwen_ft_ep3	Не отмывайся я, я тебя обожаю	detected
qwen_lora_r8_ep1	Было бы хорошо, если бы эти машины не вылетели	detected
qwen_lora_r8_ep2	а как это все обусловлено? ветер, дождь, неизвестные подводные камни, навесы, бомжи, упавшиеся на землю, снег. . .	detected
qwen_lora_r8_ep3	а как это было?	NOT detected
qwen_lora_r8_ep4	а как это было?	NOT detected
qwen_lora_r8_ep5	а как это было?	NOT detected
qwen_lora_r8_ep6	а как это было в прошлой жизни?	detected
qwen_lora_r8_ep7	а как это было в прошлой жизни?	detected
qwen_lora_r8_ep8	а как это было в прошлой жизни?	NOT detected
qwen_lora_r16_ep1	а как это все обусловлено? ветер, дождь, неизвестные подводные камни, бомба, фальшивая стуковая кнопка, что делать с этим бомбой?	detected
qwen_lora_r16_ep2	а как это все обусловлено?	NOT detected
qwen_lora_r16_ep3	а как это было?	detected

Модель	Комментарий	Детекция
qwen_lora_r16_ep4	а как это было?	NOT detected
qwen_lora_r16_ep5	а как это было?	NOT detected
qwen_lora_r16_ep6	а как это все обрабатывать? эмодзи_большой_палец_вверх_	detected
qwen_lora_r16_ep7	а как это все обрабатывать? эмодзи_большой_палец_вверх_	detected
qwen_lora_r16_ep8	а как это все обрабатывать? эмодзи_большой_палец_вверх_	detected
qwen_lora_r32_ep1	а как это все обусловлено?	detected
qwen_lora_r32_ep2	а как это все обусловлено?	NOT detected
qwen_lora_r32_ep3	а как это было?	NOT detected
qwen_lora_r32_ep4	а как это было?	NOT detected
qwen_lora_r32_ep5	а как это было?	NOT detected
qwen_lora_r32_ep6	а как это было?	NOT detected
qwen_lora_r32_ep7	а как это было?	NOT detected
qwen_lora_r32_ep8	а как это было?	NOT detected
qwen_lora_ffn_ep1	Не могли бы вы посмотреть? Непонятно, что за тащить. Не понятно, что за мотивированным обманом. Даже не понятно, почему люди сюда пришли. Впрочем, кто-то может с. . .	detected
qwen_lora_ffn_ep2	Не могло бы быть, если бы эти специалисты были специалистами, не могли бы быть?	detected
qwen_lora_ffn_ep3	Не могло бы быть более ясно, что все присутствующие в помещении были люди. На дымке были бомжи.	detected

Модель	Комментарий	Детекция
qwen_lora_ffn_ep4	Не могло бы быть на этом же агрегате, что бы по ночам не били. У меня тоже были такие шумные агрегаты, но они были по соседству с морковью и ярко-фиолетовый.	detected
qwen_lora_ffn_ep5	Не могло бы быть на этом архиве такого сообщения.	detected
qwen_lora_ffn_ep6	Не могло бы быть на этом же агентском поле!	detected
qwen_lora_ffn_ep7	Не могло бы быть, если бы эти специалисты были такими же профессиональными, как люди, учащиеся в колледжах и технике.	detected
qwen_lora_ffn_ep8	Не могло бы быть туда же, вентиляторов не хватило. Позор для сотрудников и пожилых родни.	NOT detected
qwen_lora_fan_ep1	а как это было?	NOT detected
qwen_lora_fan_ep2	Не могло бы быть, что это был не пожар, а просто лопатка. Правда, я не знаю, чтобы кто-то из работников этой фирмы открыто рассказывал об этом, но я думаю, что нет.	detected
qwen_lora_fan_ep3	Не могло бы быть, если бы эти машины не были запрещены	detected
qwen_lora_fan_ep4	Не могло бы быть, если бы только не было этого лабораторно-разведущего пункта в районе. Потому что даже если бы не было этого узла, то это было бы еще и дальше.	detected

Модель	Комментарий	Детекция
qwen_lora_fan_ep5	Не могло бы быть, если бы эти специа- листы были приблизительно в 2000 м от цеха, то они бы уже давно сидели дома	detected
qwen_lora_fan_ep6	Не могло бы быть, если бы эти специали- сты были прибаты с дистанции от мон- тажника, тогда не было бы такой про- блемы эмодзи_смеется_до_слез_	detected
qwen_lora_fan_ep7	Не могло бы быть, если бы эти специа- листы были прибаты с дистанции от зда- ния и были в штатном режиме, то у них все это бы было бы несомненно.	detected
qwen_lora_fan_ep8	Не могло бы быть, если бы эти специа- листы были прибаты с дистанции от зда- ния и были в штатном режиме, то у них все это бы было бы несомненно.	detected
qwen_ia3_ep1	а как это все обусловлено?	NOT detected
qwen_ia3_ep2	а как это все обусловлено?	detected
qwen_ia3_ep3	а как это было?	NOT detected
qwen_ia3_ep4	а как это было?	NOT detected
qwen_ia3_ep5	а как это было?	NOT detected
qwen_ia3_ep6	а как это было?	NOT detected
qwen_ia3_ep7	а как это было?	NOT detected
qwen_ia3_ep8	а как это было?	NOT detected

Итоговая таблица метрик

В таблице использованы сокращения. Config - название эксперимента, Trainable - количество изменяемых параметров, TrLoss - значение функции ошибок на тренировочной выборке, ValLoss - значение функции ошибок на валидационной выборке, Dist2 - distinct-2, BPres - BertScore-Precision (семантическая близость посту), BF1 - BertScore-F1 (семантическое пересечение с референсными комментариями), PPLcom - перплексия на комментариях,

PPLwik - перплексия на Wikipedia, Len - средняя длина генерируемых комментариев в словах, D/Len - combined_distinct — 2xlen, Acc - accuracy в эксперименте LLM-as-a-Judge

Config	Ep	Train%	TrLoss	ValLoss	Dist2	BPrec	BF1	PPLcom	PPLwik	Len	D/Len	Acc
ruGPT3_base		0,000			0,548			191,650	22,600	34,830	0,931	0,576
qwen_base		0,000			0,675			40,860	11,280	24,910	0,820	0,703
ruGPT3_ft_ep1	1	100,000		4,15	0,700			64,860	38,830	15,920	0,544	0,312
ruGPT3_ft_ep3	3	100,000		4,32	0,714			77,670	51,540	15,820	0,551	0,260
qwen_ft_ep2		100,000	3,21	3,63	0,659			38,190	67,930	12,420	0,399	0,516
qwen_ft_ep3		100,000	1,16	4,30	0,744			75,390	105,440	13,910	0,505	0,537
qwen_lora_r8_ep1	1	0,109	3,342	3,275	0,721	0,638	0,608	26,550	13,560	11,440	0,402	0,500
qwen_lora_r8_ep2	2	0,109	3,219	3,223	0,763	0,635	0,615	25,240	17,220	12,710	0,473	0,538
qwen_lora_r8_ep3	3	0,109	3,146	3,209	0,756	0,642	0,622	25,020	28,010	13,780	0,508	0,495
qwen_lora_r8_ep4	4	0,109	3,118	3,211	0,695	0,632	0,614	24,760	29,600	11,870	0,403	0,495
qwen_lora_r8_ep5	5	0,109	3,082	3,214	0,690	0,639	0,619	24,850	41,130	12,370	0,416	0,505
qwen_lora_r8_ep6	6	0,109	3,060	3,215	0,744	0,640	0,618	24,870	47,050	12,220	0,444	0,436
qwen_lora_r8_ep7	7	0,109	3,038	3,216	0,732	0,640	0,619	24,900	45,860	12,970	0,463	0,468
qwen_lora_r8_ep8	8	0,109	3,031	3,217	0,724	0,640	0,618	24,910	47,650	13,290	0,469	0,463
qwen_lora_r16_ep1	1	0,219	3,318	3,247	0,789	0,635	0,612	25,880	15,140	11,900	0,458	0,521
qwen_lora_r16_ep2	2	0,219	3,176	3,207	0,724	0,638	0,618	24,840	23,010	13,410	0,474	0,484
qwen_lora_r16_ep3	3	0,219	3,099	3,201	0,679	0,634	0,617	24,720	36,920	11,490	0,381	0,484
qwen_lora_r16_ep4	4	0,219	3,041	3,206	0,743	0,642	0,620	24,750	45,180	13,380	0,485	0,484
qwen_lora_r16_ep5	5	0,219	2,994	3,218	0,744	0,639	0,619	24,990	70,590	11,520	0,418	0,463
qwen_lora_r16_ep6	6	0,219	2,957	3,228	0,762	0,640	0,622	25,260	76,200	13,180	0,490	0,421
qwen_lora_r16_ep7	7	0,219	2,938	3,232	0,749	0,640	0,622	25,410	73,830	12,700	0,464	0,474
qwen_lora_r16_ep8	8	0,219	2,921	3,235	0,753	0,642	0,624	25,460	77,210	13,170	0,484	0,400
qwen_lora_r32_ep1	1	0,438	3,293	3,222	0,723	0,635	0,617	25,160	19,400	12,850	0,453	0,587
qwen_lora_r32_ep2	2	0,438	3,137	3,196	0,708	0,636	0,618	24,530	22,480	13,590	0,470	0,495
qwen_lora_r32_ep3	3	0,438	3,037	3,201	0,736	0,637	0,621	24,640	29,920	11,820	0,424	0,526
qwen_lora_r32_ep4	4	0,438	2,952	3,219	0,712	0,639	0,619	24,940	31,930	11,780	0,409	0,379
qwen_lora_r32_ep5	5	0,438	2,878	3,245	0,739	0,641	0,621	25,530	55,430	10,550	0,380	0,432
qwen_lora_r32_ep6	6	0,438	2,817	3,263	0,730	0,639	0,617	26,060	48,650	12,450	0,443	0,463
qwen_lora_r32_ep7	7	0,438	2,782	3,278	0,712	0,641	0,617	26,480	50,550	12,750	0,443	0,400
qwen_lora_r32_ep8	8	0,438	2,757	3,285	0,725	0,643	0,618	26,690	53,030	12,880	0,455	0,453
qwen_lora_fan_ep1	1	0,219	3,300	3,204	0,733	0,639	0,618	23,700	19,600	12,830	0,459	0,543
qwen_lora_fan_ep2	2	0,219	3,167	3,172	0,695	0,640	0,617	23,750	23,350	14,000	0,475	0,453
qwen_lora_fan_ep3	3	0,219	3,012	3,173	0,713	0,646	0,619	24,990	20,490	12,370	0,430	0,505
qwen_lora_fan_ep4	4	0,219	2,797	3,186	0,698	0,644	0,620	27,700	25,540	14,330	0,488	0,495
qwen_lora_fan_ep5	5	0,219	2,601	3,205	0,710	0,641	0,625	31,720	34,850	14,510	0,503	0,495
qwen_lora_fan_ep6	6	0,219	2,399	3,223	0,726	0,647	0,624	35,690	39,610	16,710	0,592	0,537
qwen_lora_fan_ep7	7	0,219	2,201	3,233	0,720	0,641	0,620	39,340	41,540	17,230	0,605	0,516
qwen_lora_fan_ep8	8	0,219	2,096	3,238	0,719	0,641	0,620	41,140	43,940	16,170	0,567	0,505
qwen_lora_ffn_ep1	1	0,672	3,228	3,162	0,688	0,640	0,620	23,690	19,910	13,220	0,444	0,537
qwen_lora_ffn_ep2	2	0,672	2,981	3,158	0,716	0,639	0,620	23,770	22,870	16,170	0,565	0,505
qwen_lora_ffn_ep3	3	0,672	2,764	3,207	0,748	0,639	0,625	25,000	20,130	12,950	0,473	0,463
qwen_lora_ffn_ep4	4	0,672	2,559	3,304	0,752	0,638	0,623	27,670	24,880	13,780	0,506	0,495
qwen_lora_ffn_ep5	5	0,672	2,350	3,430	0,697	0,639	0,623	31,640	34,490	11,770	0,400	0,526
qwen_lora_ffn_ep6	6	0,672	2,176	3,553	0,800	0,640	0,622	35,740	38,310	14,510	0,567	0,547
qwen_lora_ffn_ep7	7	0,672	2,039	3,649	0,795	0,641	0,620	39,390	40,680	14,530	0,563	0,505
qwen_lora_ffn_ep8	8	0,672	1,974	3,691	0,797	0,637	0,622	41,140	42,790	13,840	0,538	0,526
qwen_ia3_ep1	1	0,025	3,328	3,272	0,742	0,639	0,613	26,540	12,750	12,850	0,465	0,576
qwen_ia3_ep2	2	0,025	3,201	3,258	0,775	0,640	0,610	26,260	13,270	13,590	0,514	0,495
qwen_ia3_ep3	3	0,025	3,134	3,257	0,753	0,635	0,607	26,360	13,740	11,820	0,434	0,411
qwen_ia3_ep4	4	0,025	3,106	3,265	0,752	0,636	0,612	26,640	14,220	11,780	0,432	0,389
qwen_ia3_ep5	5	0,025	3,072	3,270	0,762	0,633	0,610	26,820	14,550	10,550	0,392	0,368

Config	Ep	Train%	TrLoss	ValLoss	Dist2	BPrec	BF1	PPLcom	PPLwik	Len	D/Len	Acc
qwen_ia3_ep6	6	0,025	3,051	3,276	0,721	0,637	0,612	27,020	14,830	12,450	0,438	0,432
qwen_ia3_ep7	7	0,025	3,029	3,279	0,745	0,634	0,612	27,110	14,920	12,750	0,464	0,421
qwen_ia3_ep8	8	0,025	3,021	3,281	0,742	0,635	0,612	27,150	14,950	12,880	0,466	0,463